There is also no standard describing how a client presents the caveat identifier to the third-party service. In practice, this limits the use of third-party caveats because client developers need to know how to integrate with each service individually, so they are typically only used within a closed ecosystem.

For an example of how third-party caveats might be used, OpenID Connect (see chapter 7) supports sending requests as encrypted JWTs (see https://openid.net/specs/openid-connect-core-1_0.html#JWTRequests). Such a provider could be implemented to return an access token in the form of a discharge macaroon, where the caveat identifier is the encrypted request and the caveat key is the OIDC nonce parameter (hidden inside the encrypted request). This would allow an API to add a third-party caveat that required users to be authenticated using a certain OIDC provider. At the present time, no OIDC provider supports this kind of flow, and few services support macaroons, so practical use of macaroons is largely confined to first-party caveats and custom service integrations. Nevertheless, macaroons are still a compelling technology and one that I expect to see wider adoption in future. In later chapters you'll learn how macaroon caveats can be used to enforce decentralized authorization for microservices and disconnected authorization in the Internet of Things.

EXERCISES

6. Which of the following apply to a first-party caveat? Select all that apply.